

ROBOTICS & ARTIFICIAL INTELLIGENCE DEPARTMENT

Total Marks: _____

Obtained Marks: _____

Computing Vision
Lab Task # 01

Submitted To: Mr. Muhammad Azhar

Student Name: Habiba Bibi

Reg. Number: 23108111

1) List at least 10 features / functional requirements of your project

Prepare a scenario/case study for your own project idea and provide feature list relevant to the proposed scenario.

ADD KEYWORDS FOR SPECIFIC SUBJECT AND PROJECT

2) Provide a feasibility study

Prepare a table mapping each requirement to available Tools/resources/APIs/packages/libraries that would help you implement.

Sr.	Requirements	Resources

3) Technical developer guide

Choose any three features from your feature list and provide a detailed, step-by-step technical guide on how to implement them.

E.g. Step by step procedure to install a required library, to configure an API, and to implement a requirement by using specific Tools/resources/APIs/packages/libraries.

4) Implementation

Using any ONE of the available tools/technology stack, implement the three selected features inside a small working project.

Submission Requirements:

- Push your completed mini project to your own GitHub repository.
- Include a short README file describing the added features and navigation flow.
- Prepare screenshots of your project code from android studio, output from emulator or separate device, and your git repository along with feasibility study to include in your submission document.

Solution:

Project Title

Real-Time Face Detection Attendance System Using OpenCV

Scenario / Case Study

A university in Rawalpindi wants to automate classroom attendance. Instead of manual roll call, a webcam installed at the classroom entrance detects a student's face and marks attendance automatically in a database. This system reduces manual effort, prevents proxy attendance, and maintains digital attendance records.

Project Keywords

Computer Vision, OpenCV, Haar Cascade, Face Detection, Image Processing, Real-Time Video Processing, SQLite Database, Attendance Automation, Python.

Functional Requirements:

1. Capture live video from webcam
2. Convert video frames to grayscale
3. Detect faces using Haar Cascade classifier
4. Draw bounding box around detected faces
5. Automatically mark attendance when face detected
6. Store student name, date, and time
7. Create database automatically if not exists
8. Prevent duplicate attendance for same date
9. Display real-time video feed window
10. Exit application using ESC key
11. Show status message on screen
12. Save attendance in local SQLite database

FEASIBILITY STUDY

Requirement Mapping Table

Sr	Requirement	Tools / Resources / Libraries
1	Webcam capture	OpenCV VideoCapture()
2	Face detection	Haar Cascade XML
3	Image processing	OpenCV cvtColor()
4	Draw rectangle	OpenCV rectangle()
5	Attendance storage	SQLite
6	Database creation	sqlite3 (Python built-in)
7	Date & time stamping	Python datetime
8	Duplicate prevention	SQL SELECT query
9	Real-time display	OpenCV imshow()
10	Exit control	OpenCV waitKey()

Technical Feasibility

The system is technically feasible because:

- OpenCV provides built-in face detection.
- SQLite is lightweight and requires no external server.
- Runs on standard laptop with webcam.

Economic Feasibility

- All tools are open-source.
- No paid API required.
- No additional hardware except webcam.

Operational Feasibility

- Easy to deploy in classrooms.
- User-friendly interface.
- Minimal training required.

TECHNICAL DEVELOPER GUIDE

(Three Selected Features)

1. Face Detection
2. Attendance Logging
3. Duplicate Prevention

FEATURE 1: Face Detection Implementation

Step 1: Install Required Library

Open Command Prompt:

```
pip install opencv-python
```

Step 2: Download Haar Cascade File

Download:

```
haarcascade_frontalface_default.xml
```

Step 3: Initialize Face Detector

```
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
```

Step 4: Detect Faces

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
faces = face_cascade.detectMultiScale(gray, 1.3, 5)
```

FEATURE 2: Attendance Logging

Step 1: Create Database

```
import sqlite3
```

```
conn = sqlite3.connect("attendance.db")
```

```
cursor = conn.cursor()
```

```
cursor.execute("""
```

```
CREATE TABLE IF NOT EXISTS attendance(  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT,  
    date TEXT,  
    time TEXT  
)
```

```
""")
```

```
conn.commit()
```

Step 2: Insert Attendance Record

```
from datetime import datetime
```

```
now = datetime.now()
```

```
date = now.strftime("%Y-%m-%d")
```

```
time = now.strftime("%H:%M:%S")
```

```
cursor.execute("INSERT INTO attendance(name,date,time) VALUES(?,?,?)",  
               ("Habiba Bibi",date,time))
```

```
conn.commit()
```

FEATURE 3: Duplicate Prevention

Step 1: Check Existing Record

```
cursor.execute("SELECT * FROM attendance WHERE name=? AND date=?",  
              ("Habiba Bibi", date))
```

```
record = cursor.fetchone()
```

Step 2: Insert Only If Not Exists

```
if record is None:
```

```
    cursor.execute("INSERT INTO attendance(name,date,time) VALUES(?,?,?)",  
                  ("Habiba Bibi",date,time))
```

```
conn.commit()
```

```

import cv2
import sqlite3
from datetime import datetime

face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')

conn = sqlite3.connect("attendance.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS attendance(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    date TEXT,
    time TEXT
)
""")
conn.commit()

cap = cv2.VideoCapture(0)
student_name = "Habiba Bibi"

while True:
    ret, frame = cap.read()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    for (x,y,w,h) in faces:
        cv2.rectangle(frame, (x,y), (x+w,y+h), (0,255,0), 2)

        now = datetime.now()
        date = now.strftime("%Y-%m-%d")

        time = now.strftime("%H:%M:%S")

        cursor.execute("SELECT * FROM attendance WHERE name=? AND date=?",
            (student_name,date))
        record = cursor.fetchone()

        if record is None:
            cursor.execute("INSERT INTO attendance(name,date,time) VALUES(?,?,?)",
                (student_name,date,time))
            conn.commit()

            cv2.putText(frame, "Attendance Recorded", (x,y-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,255,0), 2)

            cv2.imshow("Face Attendance System", frame)

            if cv2.waitKey(1) == 27:
                break

    cap.release()
    cv2.destroyAllWindows()
    conn.close()

```

FaceAttendanceSystemPublic

Pin

Watch0

main1 Branch0 Tags

Go to file

Add file

Code

23108111-makerAdd files via upload0204cfc · now3 Commits

README.md	Update README.md	2 minutes ago
app.py	Add files via upload	now
attendance.db	Add files via upload	now
haarcascade_frontalface_default.xml	Add files via upload	now

README

Face Detection Attendance System

This project is developed using the techniques of Computer Vision. It detects faces using OpenCV and marks attendance automatically.

Features

- Real-time face detection
- Automatic attendance logging
- SQLite database
- Duplicate prevention

How to Run

```
pip install opencv-python python app.py
```